Trabajo Práctico 0

> Versión 2.4.2

El TP0 es una práctica inicial para empezar a familiarizarse con algunas de las herramientas necesarias para el trabajo práctico cuatrimestral como es la configuración del entorno, el lenguaje C, etc. Es un ejercicio que sirve como base para empezar el TP luego.

El TP0 se realiza en **etapas**, cada una de ellas con un entregable que servirá de base para la siguiente. La idea de este ejercicio es que lo realicen de manera individual o grupal (no más de cinco, idealmente los mismos con los que harán el TP cuatrimestral) y tenerlo listo para cuando se presenta el Trabajo Práctico.

No es necesario tener el ejercicio completo y su realización no lleva nota, pero completar el mismo es **altamente recomendable**, ya que el entorno de desarrollo es muy similar al que recibirán a la hora de generarse el repositorio del grupo para el TP, cuyo enunciado publicaremos más adelante en el blog.

Objetivo

El objetivo de este TP0 es empezar a familiarizarse con el entorno en el que desarrollarán el TP de la materia, aprendiendo en el proceso cómo utilizar las commons por su cuenta. La idea es que siguiendo este documento logren completar las funciones vacías y comentarios que les dejamos en el código.

> **TIP**
>
> Pueden hacernos cualquier pregunta que tengan sobre el enunciado, C, o cualquier otro concepto en los medios de consulta de la práctica.

Requisitos

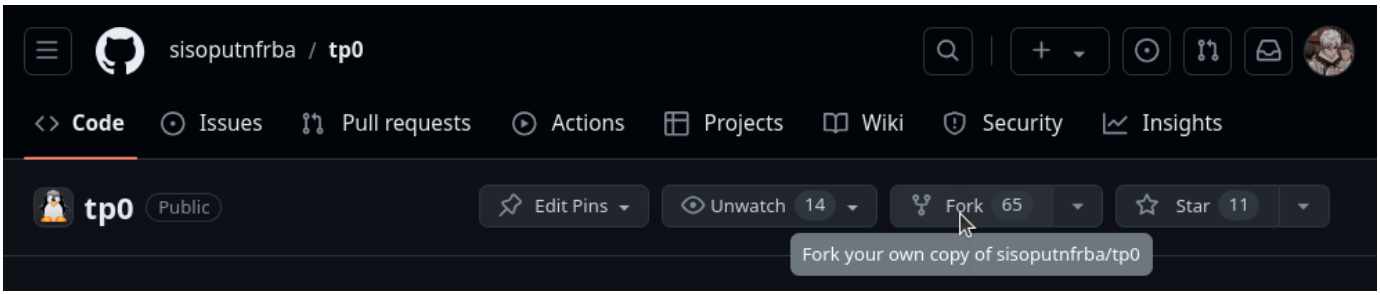
- Contar con un entorno Linux

- Tener configuradas las commons y git
- Haber leído las guías de sockets y serialización
- Tener a mano el repo del TP0

Etapa 1: Setup inicial

Forkear el repositorio

Primero, vamos a *forkear* el repo del TP0 y clonarlo en nuestra VM. Para ello, en la página de GitHub del repo hay un botón "Fork":



Que nos llevará a una página para elegir el nombre de nuestro repo forkeado, al que también llamaremos `tp0` .

> **IMPORTANTE**
>
> Cuando les toque hacer el TP cuatrimestral, **no deberán forkear el repositorio**, ya que todo el grupo estará obligado a trabajar en el repositorio provisto por la cátedra como está establecido en las Normas del Trabajo Práctico.

Configurar las credenciales de Git

Para poder clonar el repo, vamos a necesitar autenticarnos en GitHub. Existen varias formas de hacerlo, pero la que vamos a utilizar en este caso es generando un par de claves SSH y agregándolas a nuestra cuenta de GitHub.

Entonces, para generar dichas claves, vamos a abrir una terminal y ejecutar el siguiente comando:

bash

```bash
ssh-keygen -t ed25519 -C "your@email.com"
```

Donde `"your@email.com"` es el email que tienen asociado a su cuenta de GitHub.

Luego, vamos a seguir las instrucciones que nos da el comando para generar las claves. Por defecto, se van a guardar en el directorio `~/.ssh/` con los nombres `id_ed25519` y `id_ed25519.pub`.

Por último, vamos a agregar la clave pública a nuestra cuenta de GitHub moviéndonos a `Settings` > `SSH and GPG keys` > `New SSH key`.

- El tipo de clave es `Authentication Key`
- La clave es el contenido de la clave pública que generamos, el cual pueden ver con el comando `cat ~/.ssh/id_ed25519.pub`.

Una vez hecho esto, podemos verificar que todo está configurado correctamente con el comando:

bash

```bash
ssh -T git@github.com
```

IMPORTANTE

La primera vez nos va a preguntar si queremos agregar la clave a la lista de hosts conocidos:

txt

```
The authenticity of host 'github.com' can't be established.
ED25519 key fingerprint is SHA256:+asdrfadfasfsdf/asdfsdafsdafdsafdf.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Vamos a responder `yes` para agregar la clave a la lista de hosts conocidos y poder autenticarnos.

Si todo salió bien, deberíamos ver un mensaje de bienvenida de GitHub:

```
Hi TuUsuarioDeGitHub! You've successfully authenticated, but GitHub does not pr
```

Clonar el repositorio

Una vez que tengamos nuestro repo forkeado, vamos a clonarlo en nuestra VM con `git clone` copiando el link que se encuentra en la pestaña `SSH`, ya que es el método que configuramos anteriormente.

```bash
git clone git@github.com:TuUsuarioDeGitHub/tp0.git
```

Abrir desde Visual Studio Code

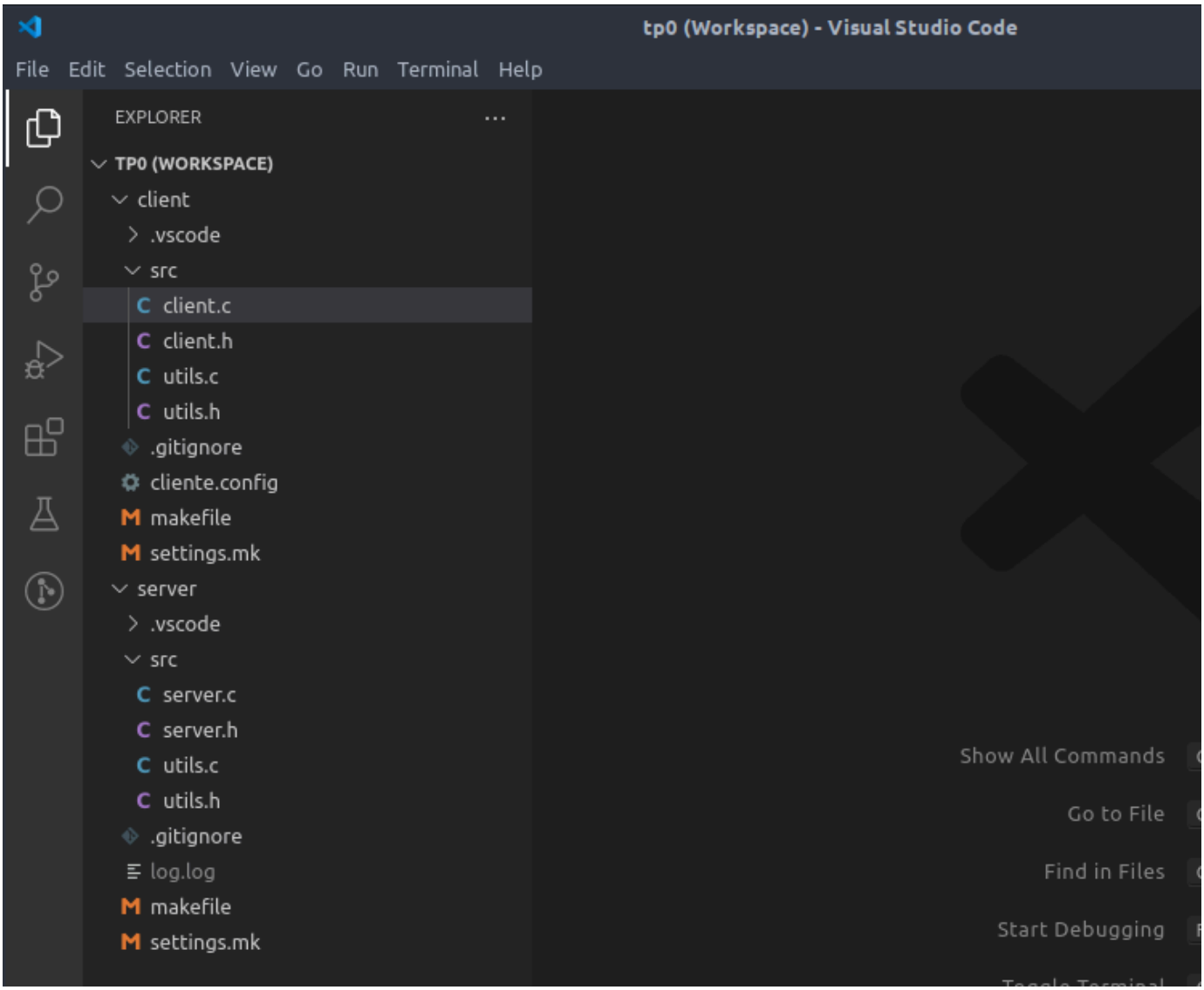
Por último, abriremos el proyecto desde Visual Studio Code utilizando el archivo `tp0.code-workspace` que se encuentra en el repo:

```bash
cd tp0
code tp0.code-workspace
```

TIP

También podemos abrir el proyecto abriendo Visual Studio Code haciendo click en `File > Open Workspace from File...` y seleccionando el archivo `tp0.code-workspace` que se encuentra en la carpeta del TP0.

Una vez abierto el workspace, vamos a ver que tenemos dos módulos: `client` y `server`, con su propio código fuente y configuraciones:



TIP

Si venís de algún cuatrimeste anterior y preferís usar Eclipse, podés seguir la guía de Eclipse.

Tené en cuenta que esta etapa va a resultar bastante más laboriosa que en Visual Studio Code, ya que la configuración debe hacerse manualmente desde cero.

Etapa 2: Comandos básicos

El objetivo de esta etapa es aprender un par de funcionalidades que utilizaremos bastante durante todo el desarrollo del trabajo práctico cuatrimestral.

Logging

Durante todo el TP iremos logueando en un archivo de texto las diferentes acciones que el programa vaya realizando, tanto las correctas, como los errores. Para ello, utilizaremos las funciones de logging que proveen las commons.

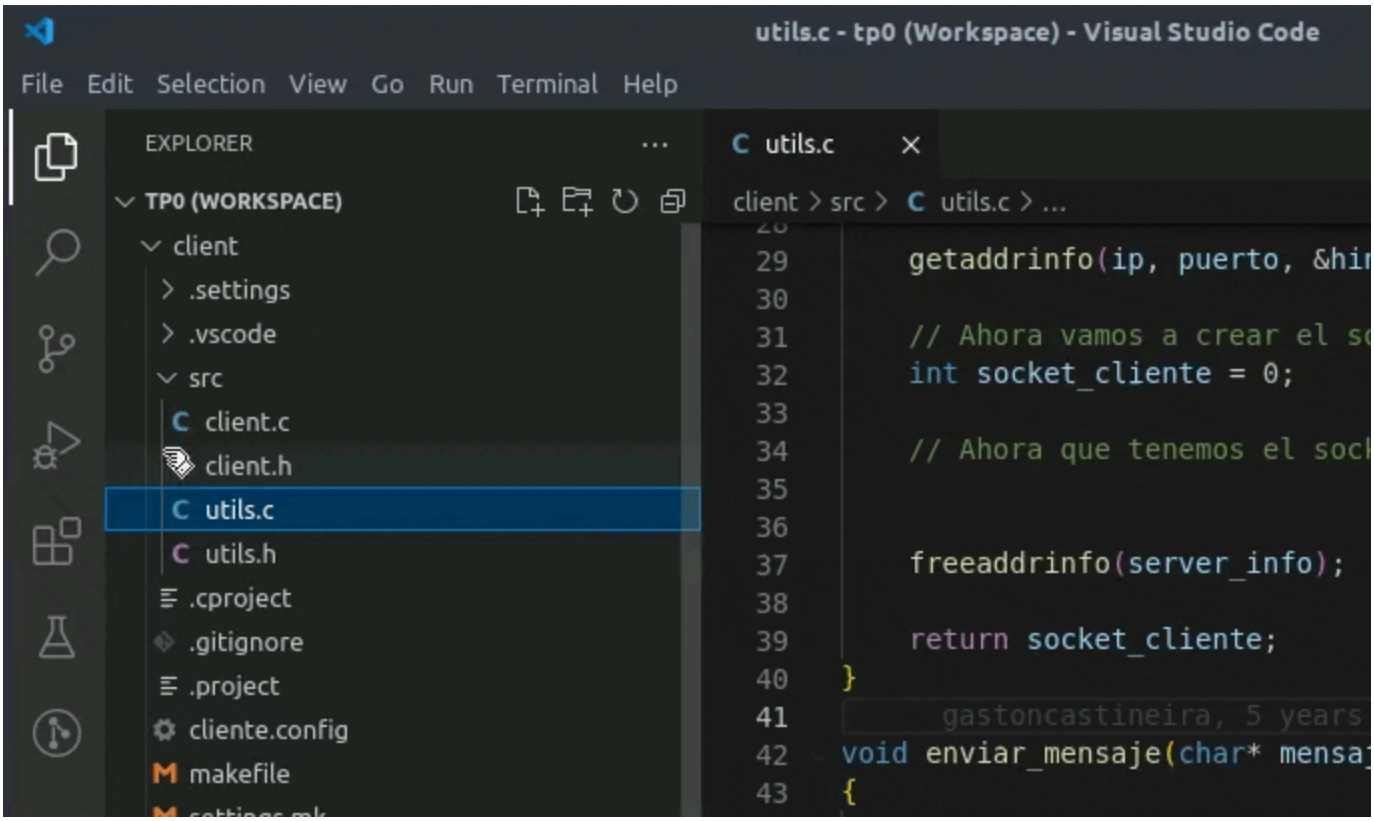
Parados en el archivo `cliente.c`, si revisamos la [documentación de log de las commons]() vamos a encontrar la función `log_create()`, que nos devuelve un logger listo para usar.

Ayudándonos con la descripción que aparece en el header, vamos a configurarla para que:

- Loguee en el archivo "tp0.log"
- Muestre los logs por pantalla (y no solo los escriba en el archivo)
- Muestre solo los logs a partir del nivel "info".

Creado nuestro logger, usemos `log_info()` para loggear el string `"Soy un Log"` y cerremos el logger al final del programa con `log_destroy()`.

Compilemos y ejecutemos el programa moviéndonos a la sección "Run and Debug" en la barra lateral izquierda, seleccionando "run (client)":



Archivos de configuración

Estaría bueno que ese valor que logueamos no esté hardcodeado en el código, sino que podamos configurarlo para que varíe sin tener que recompilar todo el proyecto, por lo que vamos a leerlo a partir de un archivo de configuración y lo vamos a loguear usando nuestro logger.

Para ello vamos a usar las config de las commons. Siguiendo su header, creemos una config que levante el archivo `"cliente.config"` y obtengamos el valor de la key `CLAVE` en formato string.

> **TIP**
>
> Para saber dónde guardar el archivo config y cómo hace Visual Studio Code para leerlo, podés consultar la guía de paths.

Usemos el logger anterior para mostrar el valor que obtuvimos. Compilamos, corramos el programa y evaluemos los resultados.

¡No se olviden de destruir el config al final del programa!

> **IMPORTANTE**
>
> Para todas las funciones de biblioteca que uses, recordá chequear los **valores de retorno** de las mismas para poder manejar los casos de error.
>
> En este caso, si llegamos a tener algún error al crear el config vamos a querer terminar con la ejecución:

c

```c
if (config == NULL) {
    // ¡No se pudo crear el config!
    // Terminemos el programa
}
```

> Para terminar el programa desde cualquier lugar, pueden utilizar la función `abort()` .

Leer de consola

De los comandos básicos, nos queda leer de consola. Si bien existen muchas formas de hacerlo, vamos a usar la biblioteca `readline` .

Necesitamos incluirla usando:

```c
#include <readline/readline.h>
```

Una vez incluida, la función `readline("> ")` va a hacer que el programa espere a que se ingrese una línea y devolverla en un string ya listo para loggear.

La misma ya se encuentra agregada en el TP0, por lo que no hace falta hacer este include.

> **IMPORTANTE**
>
> Recuerden que `readline()` no te libera la memoria que devuelve, por lo que es necesario liberarla usando `free()` .

> **TIP**
>
> Para más info sobre algunas features más avanzadas de `readline()` , pueden consultar la guía de readline.

Strings

Terminando con esta etapa, nos gustaría que el TP0 lea de consola todas las líneas que se ingresen, las loguee y, si se ingresa una línea vacía, termine con el programa.

Si ejecutamos el comando `man readline` en la consola (o visitamos el manual en internet), podemos ver en la sección "Return Value" que, ante una línea vacía, el valor de retorno de la función es un string vacío.

Pero... ¿cómo hacemos para revisar eso?

Los strings son cadenas de caracteres terminadas con `'\0'` (el caracter nulo de la tabla ASCII).

```
char str[6] = “Hello”;
```

index	0	1	2	3	4	5
value	H	e	l	l	o	\0
address	1000	1001	1002	1003	1004	1005

Eso implica que un string vacío va a tener, en su contenido, ese caracter como primer valor, por lo que podemos usar una comparación como **condición de corte**.

También podemos usar la función `strcmp()` de la biblioteca estándar de C para comparar strings. En este caso, compararíamos lo que nos devuelva `readline()` con un string vacío `""` para saber si debemos salir del bucle o no.

> **IMPORTANTE**
>
> Si en alguna etapa del TP el programa no se comporta como esperaban, pueden intentar ejecutarlo línea por línea siguiendo el tutorial de debugging en Eclipse.
>
> Si bien estamos usando Visual Studio Code, el tutorial es aplicable a cualquier IDE que quieran utilizar.

> **TIP**
>
> Las commons también proveen funciones para simplificar el manejo de strings. Pueden consultar su documentación en el sitio web.

Etapa 3: Programar el Cliente-Servidor

> **IMPORTANTE**
>
> Para poder ayudar con los conceptos y aspectos técnicos de esta estapa tienen disponible la guía de sockets.

A partir de esta etapa, vamos a plantear una arquitectura cliente-servidor. Para esta sección, ambos `client` y `server` en sus respectivas carpetas tienen un archivo `utils.c` con comentarios sobre lo que debemos hacer para poder conectar ambos procesos mediante la red.

Consigna

El entregable de esta etapa es enviar al servidor el valor de CLAVE en el archivo de configuración, y luego enviarle todas las líneas que se ingresaron por consola juntas en un paquete.

Simplificando un poco, una conexión por socket hacia otro programa va a requerir de realizar lo siguiente:

- Iniciar el servidor en la función `iniciar_servidor()` del `utils` del `server` .
- Esperar a que el cliente se conecte mediante la función `esperar_cliente()`
- Crear una conexión contra el servidor en la función `crear_conexión()` del `utils` del `client` .
- Enviar como mensaje el valor de CLAVE.
- Ir juntando las líneas que se leen por consola para luego enviarlas como paquete.
- Cerrar la conexión.

Funciones

Para simplificar el TP0, tenemos ya pre implementadas un par de funciones para comunicarnos con el servidor en el archivo fuente `utils` , que deberán consultarlo de manera similar al log y config de la etapa anterior:

- `enviar_mensaje(2)` : Recibe un mensaje en formato string y el socket a través del cual lo vamos a enviar.
- `liberar_conexion(1)` : Termina la conexión y libera los recursos que se usaron para gestionar la misma.

La única limitación es que estas funciones no nos sirven para enviar las líneas de consola todas juntas, por lo que vamos a crear un paquete. Este paquete nos va a asegurar que

toda la información que mandemos se envíe junta. Para ello, les proveemos otro conjunto de funciones o "API" para crear, rellenar y enviar paquetes:

- `crear_paquete()` : Nos crea el paquete que vamos a mandar.
- `agregar_a_paquete(3)` : Dado un **stream** y su tamaño, lo agrega al paquete.
- `enviar_paquete(2)` : Dada una conexión y un paquete, lo envía a través de ella.
- `eliminar_paquete(1)` : Elimina la memoria usada por el paquete.

> **IMPORTANTÍSIMO**
>
> Estas funciones fueron pensadas únicamente para facilitar la resolución del TP0, por lo que **no es recomendable reutilizarlas para el TP**.
>
> Para cuando necesiten enviar estructuras más complejas en el TP, está disponible la guía de serialización.

Strings vs streams

Es importante recalcar que hay que agregar al paquete un string (y no un stream). La diferencia entre ambos radica en que, si bien son una seguidilla de "bytes", el string termina en el caracter `'\0'` , mientras que un stream puede contener cualquier tipo de dato, por lo que hace falta especificar su tamaño utilizando un parámetro extra.

Para calcular el tamaño de ese string vamos a consultar la documentación de una función amiga llamada `strlen()` . Veremos que **nos devuelve el tamaño de un string, sin contar el caracter centinela**, por lo que hay que "sumarle 1".

Usando `enviar_mensaje()` para enviar nuestro valor de config y `enviar_paquete()` para enviar las líneas de consola, deberíamos poder resolver el entregable.

> **TIP**
>
> En el TP van a necesitar enviar estructuras más complejas que un string. Para hacerlo, pueden consultar nuestra guía de serialización.

Etapa 4: Desplegar en la Ubuntu Server

Por último, vamos a practicar un poco lo que vamos a hacer al momento de la entrega del TP cuatrimestral: desplegar cada uno de los módulos en forma distribuida, cada una en su propia máquina virtual.

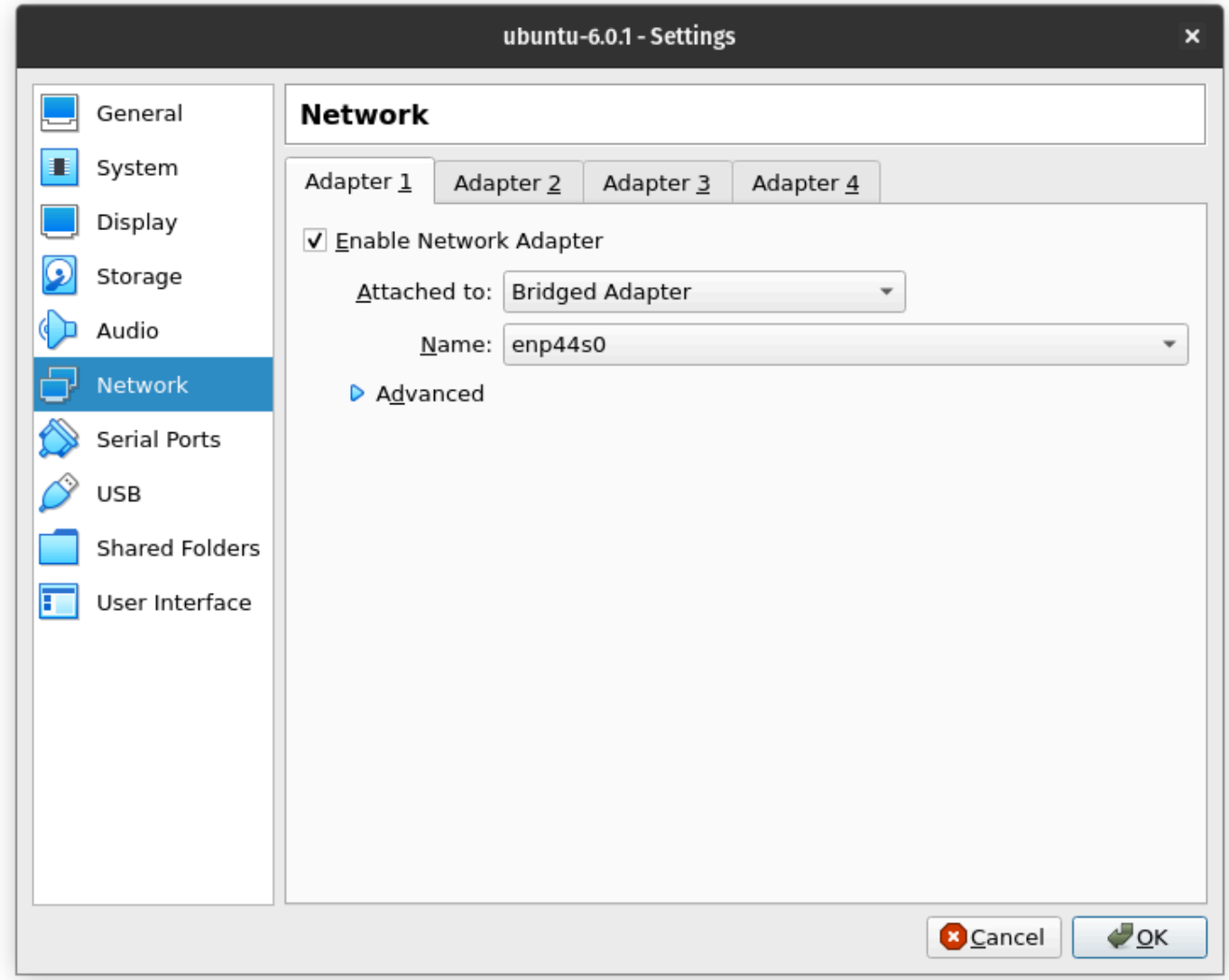
Para ello, vamos a empezar por descargar la imagen de Ubuntu Server provista por la cátedra en la sección de Máquinas Virtuales.

Estas VMs son las mismas que están instaladas en los laboratorios de la facultad, por lo que podemos estar seguros de que vamos a tener una experiencia similar a la entrega del TP.

Configuación de la red

Una vez que tengamos la VM instalada, vamos a necesitar configurar la red para que pueda comunicarse con otras máquinas virtuales en nuestra red local.

Para ello iremos a la configuración de la Ubuntu Server y en el apartado de "Network" (o "Red") elegiremos el "Bridged Adapter" (en español, "Adaptador Puente"):



Conexión por SSH

Una vez que tengamos la red configurada, podríamos iniciar la VM y utilizar la consola que nos provee VirtualBox para trabajar en ella. Sin embargo, esto no es muy cómodo ya que no podemos copiar y pegar texto, ni tampoco abrir varias ventanas.

Para solucionar esto, nos vamos a conectar a la VM por SSH. Para ello, ni bien iniciemos sesión veremos la IP que se le asignó a la VM:

```
utnso@ubuntu-server:~$ ifconfig
docker0   Link encap:Ethernet  HWaddr 02:42:9b:f5:c7:8f
          inet addr:172.17.0.1  Bcast:0.0.0.0  Mask:255.255.0.0
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

enp0s3    Link encap:Ethernet  HWaddr 08:00:27:a3:d5:84
          inet addr:192.168.0.142  Bcast:192.168.0.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fea3:d584/64 Scope:Link
          inet6 addr: 2800:810:40e:8189:a00:27ff:fea3:d584/64 Scope:Global
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:6 errors:0 dropped:0 overruns:0 frame:0
          TX packets:10 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1276 (1.2 KB)  TX bytes:1612 (1.6 KB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:176 errors:0 dropped:0 overruns:0 frame:0
          TX packets:176 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1
          RX bytes:13296 (13.2 KB)  TX bytes:13296 (13.2 KB)

Esa es la IP de la VM dentro de la red local
```

Vamos a usar esa IP, que siempre empieza con `192.168`, para conectarnos a la VM desde nuestra consola.

Tanto en Windows como en Linux, podemos abrir una consola powershell o bash y usar el comando `ssh`:

```sh
ssh utnso@192.168.XXX.YYY
```

> En XXX e YYY van los dos números que vimos antes, los cuales van entre 0 y 255.

Nos va a pedir la contraseña de la VM, que por defecto ya sabemos cuál es

Despliegue del repositorio

Para desplegar el TP0 en la VM, vamos a repetir los mismos pasos que hicimos en la guía anterior y la etapa 1, pero sin interfaz gráfica. Es decir, vamos a:

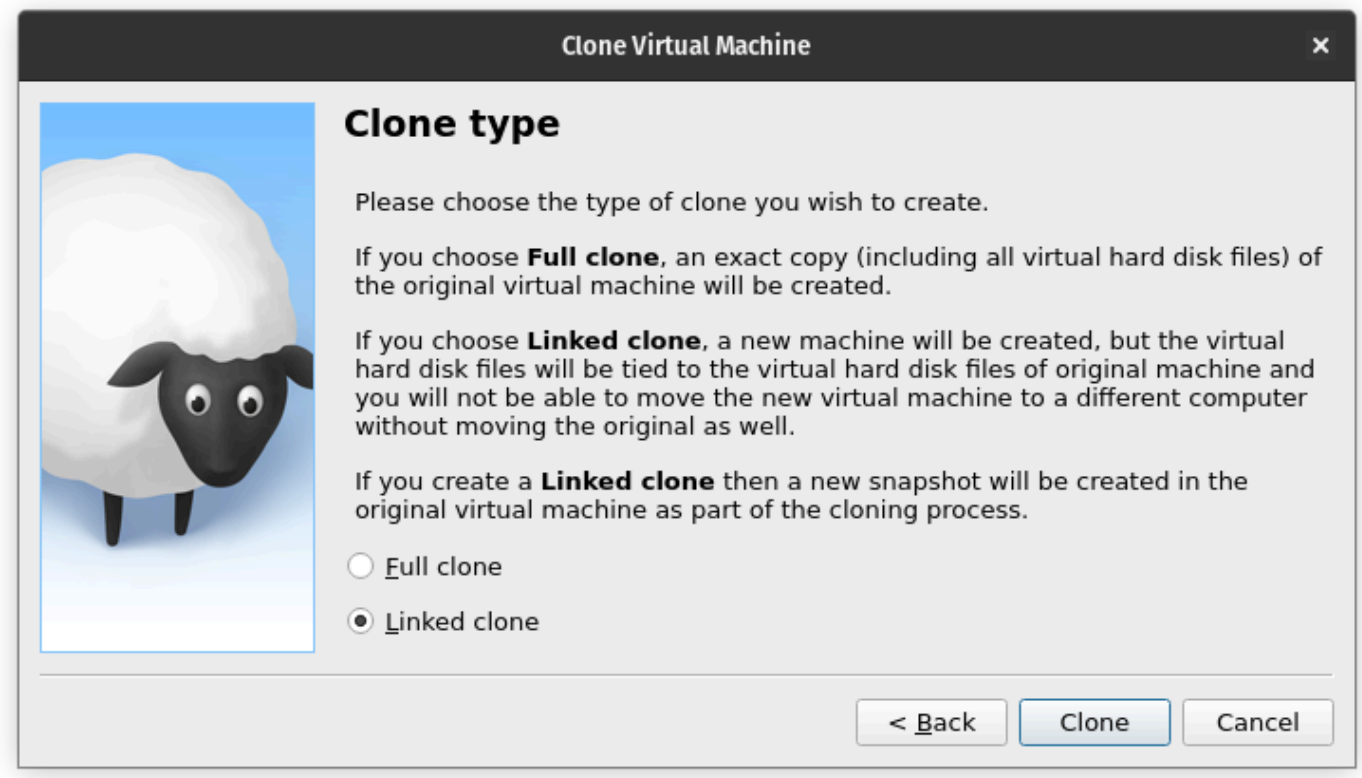
1. Instalar la so-commons-library.
2. Clonar su propio fork del TP0.
3. Moverse a la carpeta `client` y `server` y compilar el código fuente ejecutando el comando `make`.

IMPORTANTE

En este caso, como el repositorio es privado, vamos a necesitar configurar las credenciales de git para poder clonarlo. Recomendamos generar temporalmente un Personal Access Token de GitHub que podamos copiar y pegar en la consola para poder clonar el repo.

Clonar la VM

Para simular que contamos con dos máquinas distintas, lo que haremos será clonar la VM que ya tenemos configurada. Para ello, vamos a hacer click derecho en la VM > "Clone..." y seguir los pasos para clonar la VM. Recomendamos usar la opción "Linked clone" ya que el proceso de clonado es más rápido y ocupa menos espacio:



Resolver conflictos de red

Por último, vamos a iniciar sesión en una de las VMs y ejecutar las siguientes 3 líneas:

sh

```sh
sudo rm -f /etc/machine-id
sudo dbus-uuidgen --ensure=/etc/machine-id
sudo reboot
```

Esto lo que hace es generar un nuevo archivo `/etc/machine-id`, en el cual se guarda un identificador que permite al router asignarle la misma IP a una máquina cada vez que ésta se conecta a la red.

Al nosotros haber clonado la misma VM, ambas tienen el mismo `machine-id`, por lo que el router podría terminar asignándoles la misma IP a ambas VMs, lo cual generaría conflictos a la hora de conectarlas en red.

Luego de reiniciar, ejecuten `ifconfig` para corroborar que efectivamente las IPs de todas las VMs son distintas.

Configurar los módulos

Ahora sí, vamos a agregar la IP de la VM del `server` al archivo de configuración del `client` para que se puedan comunicar entre sí. Para esto pueden utilizar el editor de texto `nano`.

> **TIP**
>
> En la guía de Bash tenemos una sección dedicada a `nano` con un mini ejemplo, por si necesitan ayuda sobre cómo usarlo.

Notas finales

El transcurso de esta guía de primeros pasos fue un poco largo, ¡pero aprendimos un montón! Recapitulemos un poco:

- Pudimos configurar nuestro entorno de desarrollo.
- Aprendimos a usar funciones de las commons que nos van a ser muy útiles.
- Aprendimos sobre reservar memoria, liberarla y leer por consola.

- Pudimos mandar mensajes por red a otro programa.
- Aprendimos cómo desplegar el TP en una VM sin interfaz gráfica.

Esto fue todo, pero recuerden que el TP0 es solo una introducción a todas las herramientas que podemos usar.

Por lo tanto, les pedimos que consulten las guías y video tutoriales linkeados en la sección Guías de esta página para mejorar constantemente y llegar bien holgados a fin de cuatrimestre.

¡Hasta la próxima amigos!